

time out

Backend and Frontend Timeouts for GP (Shazam-like Application)

Status In Progress

JS Language JavaScript

This document outlines various timeout mechanisms and their implementation in our GP (Shazam-like) application.

Table of Contents

- [What are Timeouts?](#)
 - [Types of Timeouts](#)
 - [Practical Examples](#)
 - [Key Benefits](#)
 - [Future Considerations](#)
 - [Contributing](#)
-

1. What are Timeouts?

Timeouts refer to a period after which a system operation is terminated if it hasn't completed. They are essential for managing system performance, preventing resource exhaustion, and ensuring that unresponsive processes do not block other operations.

Common Use: Applied to client-server communications, database queries, and system processes.

2. Types of Timeouts in Frontend-Backend Communication

a. Connection Timeout

- Occurs when a client tries to establish a connection with the server.
- **Usage in GP:** When our Shazam-like app connects to the database or an API (like for audio fingerprinting or user authentication), the connection should be timely.
- **Example:** Setting connection timeouts in Node.js for the database or API requests ensures that the app does not hang if the external server is slow to respond.

b. Request Read Timeout (Server Read Timeout)

- Defines how long the server waits to receive the request from the client.
- **Usage in GP:** When uploading audio files or submitting large data, the backend may need to implement request read timeouts to prevent slow uploads from affecting performance.
- **Example:** If a user uploads an audio clip for song recognition and the upload is too slow, the server should cancel the request after a set period.

c. Wait Timeout

- When the server queues requests, a wait timeout ensures that the request is not kept waiting indefinitely before processing.
- **Usage in GP:** Useful if our Shazam-like app scales and processes many requests simultaneously. If the system is overloaded, incoming requests can be queued, and wait timeouts will ensure that they don't wait indefinitely.
- **Example:** Implementing wait timeouts in a task queue or load balancer when there's high traffic in our app.

d. Processing Timeout (Usage Timeout)

- Limits the time a request is allowed to be processed by the server.

- **Usage in GP:** During audio fingerprinting, processing the audio clip could take time depending on the algorithm used. Set a timeout to prevent overly long computations from affecting server responsiveness.
- **Example:** If fingerprint matching takes too long, the system should stop the process and return an error, prompting the user to retry.

e. Response Timeout

- Defines how long the client will wait for a response from the server before canceling the request.
- **Usage in GP:** If the server (backend) takes too long to recognize the song or respond with results, the client (frontend) may give up waiting.
- **Example:** The user uploads an audio clip for recognition, and the frontend sets a response timeout. If the server doesn't respond within a certain time (e.g., 10 seconds), the client shows an error or retry option.

3. Practical Examples for GP

a. Setting Timeouts in Node.js (Backend)

In our backend (Node.js), we can set timeouts for both HTTP requests and database connections.

<details>

<summary>Node.js Timeout Example</summary>

```
// Example: Timeout for external API request (e.g., fetching
song metadata)
const axios = require('axios');

axios.get('<https://example.com/song>', { timeout: 5000 })
// 5 seconds timeout
  .then(response => {
    console.log(response.data);
  })
  .catch(error => {
```

```

    if (error.code === 'ECONNABORTED') {
      console.error('Request timed out');
    } else {
      console.error(error.message);
    }
  });
});

```

</details>

Why Use It in GP?

When making requests to external services (e.g., an external song recognition API), timeouts ensure that if the service is slow or unresponsive, our app won't hang indefinitely. This is crucial for maintaining a smooth user experience.

b. Handling Timeouts with SQL (Database)

In our app, we will also query the database (e.g., to fetch or store audio fingerprints).

<details>

<summary>SQL Database Timeout Example</summary>

```

const mysql = require('mysql2/promise');

async function queryDatabase() {
  const connection = await mysql.createConnection({ host: 'localhost', user: 'root', database: 'shazam', connectTimeout: 10000 }); // 10 seconds timeout

  try {
    const [rows, fields] = await connection.query('SELECT * FROM songs WHERE id = ?', [songId]);
    console.log(rows);
  } catch (err) {
    if (err.code === 'PROTOCOL_SEQUENCE_TIMEOUT') {
      console.error('Query timed out');
    } else {

```

```

        console.error('Query failed', err.message);
    }
}
}

queryDatabase();

```

</details>

Why Use It in GP?

We may perform complex database queries (like matching an audio fingerprint). Setting a timeout ensures the query doesn't take too long to process, protecting our server from becoming unresponsive during heavy loads.

c. Implementing Timeouts in Express (Backend Web Server)

For server request and response handling, timeouts can be applied as middleware in Express.js.

<details>

<summary>Express.js Timeout Example</summary>

```

const express = require('express');
const app = express();

app.use((req, res, next) => {
    res.setTimeout(10000, () => { // 10 seconds
        console.log('Request has timed out.');
```

```

        res.status(408).send('Request timeout');
    });
    next();
});

app.get('/', (req, res) => {
    res.send('Song recognition result');
});

```

```
app.listen(3000, () => console.log('Server is running on port 3000'));
```

</details>

Why Use It in GP?

This prevents the server from waiting indefinitely on requests (e.g., waiting too long for an audio file upload). If the timeout is exceeded, the server can return an error response and free up resources for other users.

4. Key Benefits of Timeouts in the Project

- **Prevents Resource Exhaustion:** Timeout mechanisms free server resources by terminating unresponsive or long-running operations, preventing crashes.
- **Enhances User Experience:** Users are informed when a process takes too long, and they can retry or see a meaningful error message rather than waiting indefinitely.
- **Improves Scalability:** By preventing bottlenecks (e.g., in audio processing or database access), timeouts help the system handle more users efficiently.
- **Mitigates Attacks:** Timeouts protect against attacks like slowloris by disconnecting clients that intentionally slow down communication.